

WC2026 Dashboard — Architecture & Documentation

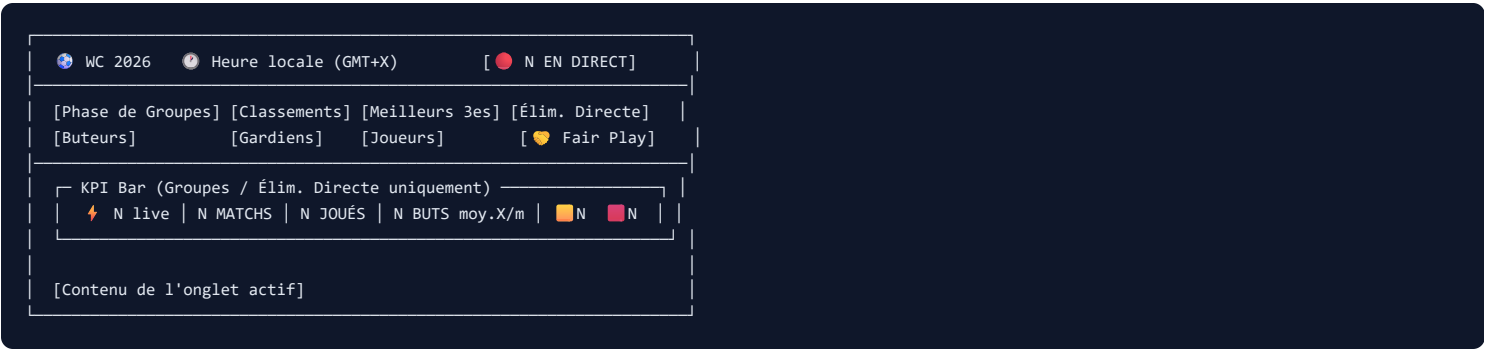
Description fonctionnelle

Vision d'ensemble

Le **Dashboard WC2026** est une application web temps réel de suivi de la Coupe du Monde 2026 (USA / Canada / Mexique, 48 équipes, 104 matchs). Accessible depuis n'importe quel navigateur, sans installation, elle offre :

- Le suivi des matchs **en direct** (scores live, minute de jeu)
- Les **classements de groupe** mis à jour en temps réel
- Des **prédictions de score** basées sur un modèle Poisson avec prior FIFA
- Les **statistiques individuelles** des joueurs agrégées depuis ESPN
- Le classement **Fair Play** par cartons
- L'adaptation automatique des **horaires au fuseau local** de l'utilisateur

Interface utilisateur — Navigation



Onglets et contenu fonctionnel

Phase de Groupes (groups)

Affiche le calendrier complet des 72 matchs de phase de groupes, organisé par date. Pour chaque match :

- Statut** : **NS** (à venir), score live avec minute de jeu, ou score final **FT**
- Heure** : convertie dans le fuseau horaire local du navigateur (si différent du Maroc)
- Prédiction** : score prédit  X-Y avec probabilités V / N / D (modèle Poisson + prior FIFA)
- Boutons Stats / Compos** sur les matchs terminés (données ESPN via Worker KV)
- Filtre groupe** : boutons A–L pour filtrer les matchs par groupe

Source : matchs depuis `football-data.org` → cache KV Worker → `allMatches` . Scores live depuis ESPN scoreboard → Worker `/data/live` .

Classements (standings)

Classements par groupe (Pts / J / V / N / D / BP / BC / DB), mis à jour en temps réel.

Source : `football-data.org` standings → cache KV Worker → `standings` .

Meilleurs 3èmes (thirds)

Liste des 12 troisièmes de groupe avec leur bilan, les 8 qualifiés mis en évidence (règle FIFA). Gestion des ex æquo (badge `ex æq.`), indicateur provisoire `prov.` si le groupe n'est pas terminé.

Source : calculé depuis `allMatches` et `standings` en mémoire locale.

Élimination Directe (knockout)

Vue liste ou bracket visuel des 32 matchs KO (32es → finale). Les placeholders (`1er Gr.A` , `V M73` ...) sont résolus dynamiquement depuis les classements. Les matchs à venir affichent une prédiction de score.

Source : structure statique `fallback.js` + résolution dynamique depuis `standings` .

 Buteurs (`scorers`)

Top 20 des buteurs du tournoi avec nombre de buts et équipe.

Source : `football-data.org scorers` → `cache KV Worker` → `scorers` .

 Gardiens (`keepers`)

Classement des gardiens par `saves` cumulées (matchs joués, buts encaissés, clean sheets).

Source : pipeline stats ESPN → KV `match:*` → Worker `/stats/keepers` .

 Joueurs (`players`)

Classement des joueurs par note moyenne pondérée (algo Rating v8). Colonnes : MJ, Min cumulées, Buts, Assists, Note.

Source : pipeline stats ESPN → KV `match:*` → Worker `/stats/players` .

 Fair Play (`fairplay`)

Classement des équipes par fair play : score `FP = jaunes + 3 × rouges` croissant. Les 5 meilleures équipes sont affichées en vert.

Source : stats ESPN par joueur (KV `match:*`) agrégées par équipe → Worker `/stats/fairplay` .

KPI Bar

Affichée en haut des onglets **Phase de Groupes** et **Élimination Directe**, la barre KPI présente les statistiques agrégées de la phase active :

Indicateur	Description
 N EN DIRECT	Matchs actuellement en cours (affiché seulement si > 0)
N MATCHS	Total des rencontres de la phase
N JOUÉS	Matchs terminés (FT)
N BUTS + moy./m	Total des buts marqués et moyenne par match joué
 N JAUNES + moy./m	Cartons jaunes totaux (depuis <code>/stats/fairplay</code>)
 N ROUGES + moy./m	Cartons rouges totaux (depuis <code>/stats/fairplay</code>)

Les cartons KO affichent 0 tant qu'aucun match KO n'est terminé dans ESPN stats.

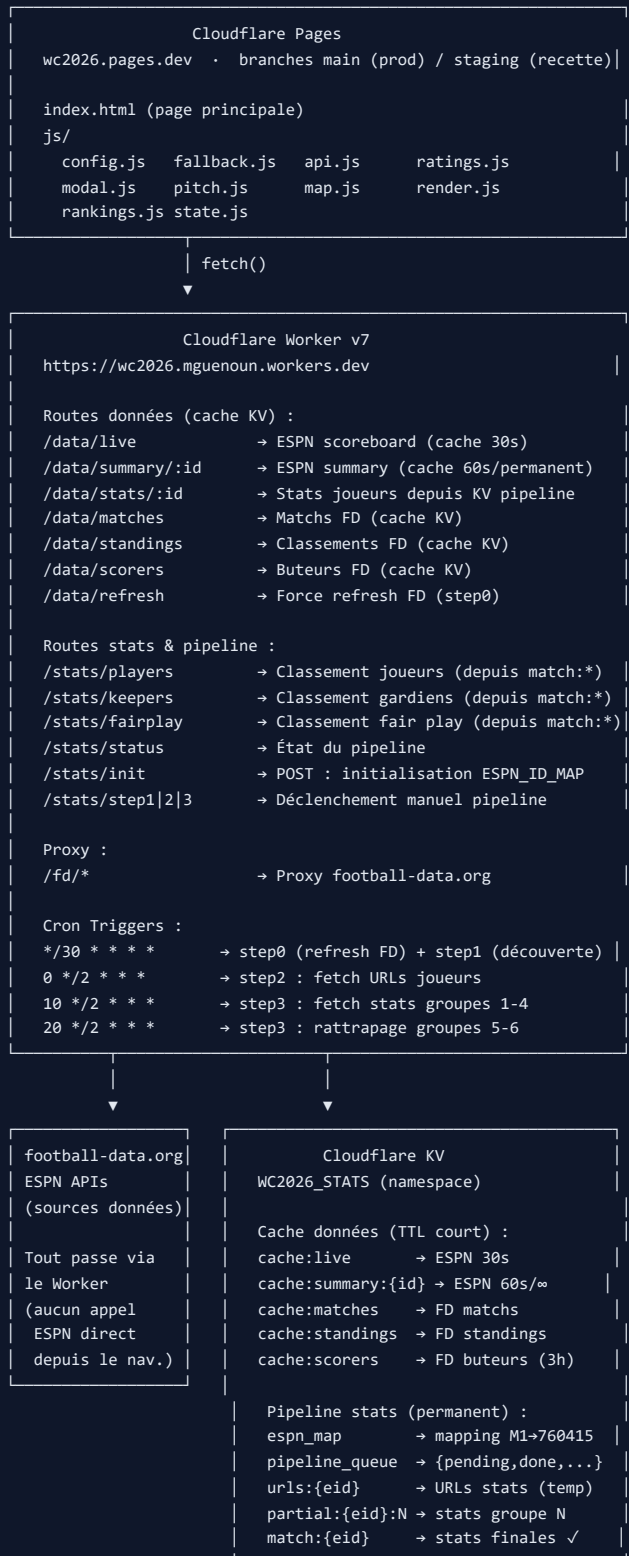
Fuseau horaire automatique

L'heure locale du navigateur est auto-détectée via `Intl.DateTimeFormat().resolvedOptions().timeZone` (variable globale `USER_TZ`). Les horaires des matchs (champ `utcDate` de `football-data.org`) sont convertis et affichés dans ce fuseau. Le fuseau détecté est affiché dynamiquement dans l'en-tête. Le regroupement par journée reste en heure Maroc (UTC+1, `DISPLAY_TZ`) pour la cohérence du calendrier.

Vue d'ensemble

Dashboard de suivi de la Coupe du Monde 2026 (USA/Canada/Mexique), déployé en tant que site statique sur **Cloudflare Pages** (branches `main` → prod, `staging` → recette) avec un backend serverless sur **Cloudflare Workers** (v7) et un cache persistant **Cloudflare KV**.

Architecture globale



Composants GitHub Pages

Structure des fichiers

```
wc2026/
├─ index.html          # Page principale (Cloudflare Pages)
├─ js/
│   ├── config.js      v3  # Constantes, maps, utilitaires, USER_TZ, localTime()
│   ├── fallback.js    v3  # Données statiques matchs (63 matchs + KO)
│   ├── api.js         v12 # Fetch Worker, traitement scores live, fetchFairPlay()
│   ├── predictions.js v5  # Modèle Poisson v4 (prédictions matchs à venir)
│   ├── ratings.js     v4  # Algo rating joueurs v8 (sans xG/xA)
│   ├── modal.js       v7  # Modales stats match + lien YouTube par but
│   ├── pitch.js       v4  # Terrain SVG, compositions, openLineupESPN
│   ├── map.js         v7  # Affichage stade (OpenStreetMap + Google Maps)
│   ├── render.js      v13 # Liste matchs, prédictions, YouTube, bracket KO, Meilleurs 3èmes, Fair Play, KPI bar
│   ├── rankings.js    v3  # Gardiens, buteurs, classement joueurs (avec Min)
│   └─ state.js        v8  # renderAll, switchView (8 vues), scheduleRefresh, init fuseau
```

Ordre de chargement des scripts (critique)

```
<script src="js/config.js?v=3"></script>      <!-- 1. Constantes globales, USER_TZ, localTime() -->
<script src="js/fallback.js?v=3"></script>     <!-- 2. Données statiques -->
<script src="js/api.js?v=12"></script>         <!-- 3. Fetch Worker/FD, fetchFairPlay() -->
<script src="js/predictions.js?v=5"></script>  <!-- 4. Modèle Poisson v4 -->
<script src="js/ratings.js?v=4"></script>      <!-- 5. Algo rating v8 -->
<script src="js/modal.js?v=7"></script>        <!-- 6. Modales + YouTube par but -->
<script src="js/pitch.js?v=4"></script>        <!-- 7. Terrain SVG -->
<script src="js/map.js?v=7"></script>          <!-- 8. Affichage stade (openMap) -->
<script src="js/render.js?v=13"></script>      <!-- 9. Liste matchs, prédictions, YouTube, bracket KO, Meilleurs 3èmes, Fair Play, KPI bar -->
<script src="js/rankings.js?v=3"></script>     <!-- 10. Classements joueurs -->
<script src="js/state.js?v=8"></script>       <!-- 11. Init + orchestration (8 vues), init fuseau -->
```

Variables globales clés (config.js)

Variable	Description
PROXY_BASE	URL du Worker (<code>https://wc2026.mguenoun.workers.dev</code>)
ESPN_ID_MAP	Mapping matchKey → ESPN event ID (ex: <code>'M1': '760415'</code>)
TEAM_MAP	Noms d'équipes ESPN → noms français
GC	Couleurs par groupe (<code>{A: '#color', B: '#color', ...}</code>)
allMatches	Array des matchs (peuplé par <code>loadFallback()</code>)
standings	Classements par groupe (peuplé par <code>fetchAll()</code>)
scorers	Liste des buteurs (peuplé par <code>fetchAll()</code>)
fairplayData	Array équipes avec <code>{team, yc, rc}</code> (peuplé par <code>fetchFairPlay()</code>)
fairplayLoaded	Boolean garde-fou pour le lazy-load fair play
activeFilter	Filtre groupe actif ('all' ou lettre)
_currentMatch	Match actuellement ouvert en modale
DISPLAY_TZ	Timezone d'affichage fixe (<code>Africa/Casablanca</code>) — groupes par jour, TODAY_STR
USER_TZ	Timezone du navigateur (auto-déTECTÉ via <code>Intl.DateTimeFormat</code>) — heures des matchs

Cloudflare Worker v7 (worker.js)

Step 0 — Refresh données FD (toutes les 30min avec step1)

- Fetch matchs FD → écrit `cache:matches` (différentiel, seulement si changement)

- Si changement détecté → fetch standings FD → écrit `cache:standings`
- Fetch buteurs FD toutes les 3h → écrit `cache:scorers`

Routes cache KV (front → Worker → KV)

Route	Source	TTL
<code>/data/live</code>	ESPN scoreboard J-1 + J	30s
<code>/data/summary/:id</code>	ESPN summary	60s si live, permanent si terminé
<code>/data/stats/:id</code>	KV <code>match:{id}</code> (pipeline)	Permanent
<code>/data/matches</code>	KV <code>cache:matches</code>	~30min (step0)
<code>/data/standings</code>	KV <code>cache:standings</code>	~30min si changement (step0)
<code>/data/scorers</code>	KV <code>cache:scorers</code>	3h (step0)
<code>/stats/fairplay</code>	KV <code>match:*</code> agrégé par équipe	Calculé à la demande

Pipeline de calcul des stats joueurs

Le plan gratuit Cloudflare limite chaque invocation à **50 sous-requêtes HTTP**. Le pipeline découpe le traitement en 3 étapes :

Step 1 — Découverte (~8 sous-requêtes)

- Scan ESPN scoreboards sur les 7 derniers jours
- Détecte les nouveaux matchs terminés → alimente `espn_map`
- Compare avec `pipeline_queue` → ajoute les ESPN IDs non traités dans `queue.pending`

Step 2 — Fetch URLs (~7 sous-requêtes)

- Prend le premier match `pending`
- Fetch ESPN summary (1) + competitors (1) + 2 rosters (2×2)
- Extrait les URLs de stats individuelles (22-52 joueurs)
- Stocke dans `KV["urls:{eid}"]`
- Déplace en `queue.processing`

Step 3 — Fetch stats par groupes de 10 (~12-42 sous-requêtes)

- Lit `KV["urls:{eid}"]`
- Fetch les stats de 10 joueurs en SÉQUENCE (pas Promise.all)
- Maximum 4 groupes par invocation (40 appels max, sous la limite de 50)
- Stocke dans `KV["partial:{eid}:{N}"]`
- Quand tous les groupes sont prêts → assemble dans `KV["match:{eid}"]`

KV Keys

Clé	Contenu	Durée de vie
cache:live	Events ESPN scoreboard J-1+J	30s
cache:summary:{eid}	ESPN summary complet	60s (live) / permanent (FT)
cache:matches	Matches FD (différentiel)	Mis à jour par step0 (~30min)
cache:standings	Classements FD	Mis à jour par step0 si changement
cache:scorers	Buteurs FD top 20	Mis à jour toutes les 3h
espn_map	Mapping matchKey → ESPN ID + infos	Permanent
pipeline_queue	{ pending:[], processing:[], done:[] }	Permanent
urls:{eid}	URLs stats joueurs + infos match	Temporaire (supprimé après assemblage)
partial:{eid}:{N}	Stats du groupe N de joueurs	Temporaire (supprimé après assemblage)
match:{eid}	Stats finales de tous les joueurs	Permanent

Secrets (Cloudflare Dashboard → Settings → Variables and Secrets)

Nom	Description
API_TOKEN_FD	Clé API football-data.org
API_TOKEN	Clé API football-data.org (alias legacy)

KV Namespace Binding

Variable name	KV Namespace	Namespace ID
STATS_KV	WC2026_STATS	749700d94d114156a758b14e2e9b6587

Cron Triggers

Expression	Fréquence	Rôle
* / 30 * * * *	Toutes les 30min	step0 (refresh FD) + step1 (détection matchs)
0 * / 2 * * * *	Toutes les 2h	step2 : fetch URLs du prochain match pending
10 * / 2 * * * *	Toutes les 2h + 10min	step3 : fetch stats groupes 1-4
20 * / 2 * * * *	Toutes les 2h + 20min	step3 : rattrapage groupes 5-6

Sources de données

Source	Usage	Accès
ESPN site.api.espn.com	Scores live, compositions, stats match	Via Worker /data/live , /data/summary/:id
ESPN Core sports.core.api.espn.com	Stats détaillées joueurs (pipeline)	Direct depuis Worker (step2/step3)
football-data.org	Matches, standings, buteurs	Via Worker /fd/* + cache KV

Aucun appel ESPN n'est fait directement depuis le navigateur. Tout passe par le Worker, qui maintient un cache KV pour limiter les requêtes upstream.

Algo Rating Joueurs (v8)

Formule par poste avec architecture base + volScore + offScore :

- **Base** : 6.3 pour tous les postes
- **Per90 atténué** : $1 + (90/\text{min} - 1) \times 0.2$ (évite la distorsion sur peu de minutes)
- **progCarries** plafonné à 6 avant normalisation
- **volScore** (métriques de volume/défense) : plafonné selon poste (1.2 CM → 1.8 DEF/FB/AM)
- **offScore** (buts, assists, tirs cadrés) : plafonné à 2.5 (2.8 pour FW)
- **Carton rouge** : -1.0 / **Carton jaune** : -0.3
- **Plage** : [4.0 — 9.5]
- **Sans xG/xA** : supprimés en v8 (causaient une inflation de +0.1 à +0.5 pts sur AM/FW vs SofaScore/FotMob)

Poids par rôle :

Rôle	Vol principal	Off principal	Pénalité
GK	saves×0.35, cs×0.30	goals×2.0	—
DEF	tackles×0.15, duels×0.80, cs×0.35	goals×1.8	—
FB	crosses×0.15, duels×0.65, cs×0.25	goals×1.6, assists×1.0	—
DM	tackles×0.18, passes×0.60	goals×1.5	—
CM	passes×0.45, duels×0.45	goals×1.4, shots×0.15	—
AM	shots×0.20, progCarries×0.08	goals×1.4, assists×0.90	-0.15 si shots<0.5
FW	—	goals×1.5, shots×0.22, assists×0.80	-0.20 si shots<0.5

Prédictions de Score — Modèle Poisson v4 (predictions.js)

Les prédictions sont calculées côté navigateur pour chaque match **à venir** (non démarré, non terminé, sans placeholder KO).

Prior FIFA

Chaque équipe dispose d'un **rating FIFA officiel** (juin 2026, 48 équipes). En l'absence de stats de tournoi, le modèle revient à ce prior :

```
priorGF_A = avgGF × (fifaA / avgFifa)^1.5
priorGA_A = avgGA × (avgFifa / fifaA)^1.5
```

- **avgGF** / **avgGA** : buts par équipe par match calculés depuis les classements en cours
- **avgFifa** ≈ 1580 (moyenne des 48 équipes)
- Exposant 1.5 : différencie suffisamment sans exagérer (Maroc vs Écosse → ~58% favori)

Régression bayésienne K=6

Après chaque match joué, les stats réelles ne pèsent que $w = \text{played} / (\text{played} + K)$:

```
w = played / (played + 6)
gf_A = w × rawGF_A + (1-w) × priorGF_A
ga_A = w × rawGA_A + (1-w) × priorGA_A
```

Avec K=6, après 1 match joué le poids sur les stats réelles est seulement 14%. Cela évite qu'une défaite difficile (ex: 0-2 vs Brésil J1) écrase le prior d'une équipe de niveau mondial.

Paramètres Poisson

```
lambdaA = clip(gf_A × ga_B / avgGF, 0.2, 4.0)
lambdaB = clip(gf_B × ga_A / avgGF, 0.2, 4.0)
```

La distribution de Poisson bivarié $P(i \text{ buts A, } j \text{ buts B}) = \text{PMF}(\lambda_A, i) \times \text{PMF}(\lambda_B, j)$ est calculée sur une grille 8×8 (0 à 7 buts chaque côté).

Argmax par catégorie

Les probabilités sont séparées en trois catégories :

- **V (Victoire A)** : somme des P(i,j) avec i > j
- **N (Nul)** : somme des P(i,j) avec i = j
- **D (Défaite A)** : somme des P(i,j) avec i < j

Le **score affiché** est le score (i,j) maximisant P dans la catégorie la plus probable (pas le score global le plus probable, ce qui permettrait d'afficher un nul 0-0 même quand V=55%).

Affichage (render.js v11)

Pour chaque match à venir, la ligne affiche :

-  X-Y en orange (stats disponibles) ou gris (prior FIFA uniquement)
- Vxx% · Nxx% · Dxx% colorisés : vert si favori, rouge si perdant, gris si nul prédit
- Tooltip : Poisson λ=X.X / Y.Y

buildPredictions() est appelé depuis api.js après chaque fetchAll() et après chaque mise à jour de standings en live.

Classement Joueurs (rankings.js v3 + Worker /stats/players)

Pipeline de calcul (Worker)

handlePlayers() agrège toutes les clés match:{eid} du KV pour construire le classement :

```
Pour chaque joueur sur tous les matchs :
totalMinutes += minutes_jouées_ce_match
totalRating += rating_ce_match × minutes_jouées_ce_match
goals += buts
assists += passes décisives
matches += 1

rating_final = totalRating / totalMinutes (moyenne pondérée par minutes)
```

Le classement est trié par rating_final décroissant.

Algo Rating v8 (ratings.js)

Formule par poste : rating = 6.3 + volScore + offScore

Métrique	Description
Per90 atténué	1 + (90/min - 1) × 0.2 — atténue la distorsion sur peu de minutes
volScore	Métriques de volume/défense (tackles, passes, duels, saves, CS) — plafonné 1.2→1.8 selon poste
offScore	Buts, assists, tirs cadrés — plafonné à 2.5 (2.8 FW)
Carton rouge	-1.0
Carton jaune	-0.3
Plage finale	[4.0 — 9.5]

Sans xG/xA (supprimés en v8 car causaient une inflation de +0.1 à +0.5 sur AM/FW vs SofaScore).

Colonnes affichées

Colonne	Description
MJ	Matches joués (cumul)
Min	Minutes cumulées sur tous les matchs
	Buts totaux (cumul)
→	Passes décisives totales (cumul)
Moy.	Note moyenne pondérée par minutes — critère de classement

Refresh des scores live

state.js — scheduleRefresh() toutes les 30s (setTimeout récursif) :

```
Tick N :
  hasLive = allMatches.some(m => m.isLive)    ← évalué au moment du tick
  Si hasLive      → fetchESPNLiveScores()      [Worker /data/live, cache 30s]
  Si tick%5 == 0  → fetchAll()                  [refresh complet toutes les 2.5min]
  Sinon tick%10==0 → fetchAll()                 [refresh complet toutes les 5min]
```

processMatches() — gestion décalage UTC/heure locale :

- Matching ESPN events sur dayKey du match (heure locale Paris)
- Fallback J-1 si le match était après minuit UTC la veille (ex: match 21h Paris = J-1 UTC)

Bracket KO et Meilleurs 3èmes (render.js / fallback.js)

- fallback.js : données statiques des 63 matchs (GS + KO), placeholders 1er Gr.X , 2e Gr.X , 3e A/B/C/D , V Mxx , Vainq./Perdant
- resolveKOTeam(placeholder) : résout les placeholders depuis allMatches et standings
- Noms résolus affichés entre parenthèses dans la vue liste et dans le bracket (tous deux dans render.js)
- getAll3rd() : collecte le 3e de chaque groupe ayant joué ≥ 1 match, trié par pts/DB/BP. Détecte les cas ex æquo (2e et 3e avec stats identiques) et renseigne un champ coTeam .
- buildThirdAssign() : matching bipartite (chemin augmentant) pour assigner chaque 3e qualifié à un slot KO. Chaque slot liste les groupes éligibles selon la règle FIFA ; l'algo garantit un matching maximal même quand le greedy simple échouerait.
- renderThirds() : onglet dédié affichant les 12 troisièmes classés, top 8 qualifiés mis en évidence (✓), badge prov. si le groupe n'a pas fini, badge ex æq. si deux équipes sont parfaitement à égalité.
- En cas d'ex æquo, la valeur dans la Map _thirdAssign contient les deux noms (ex: "Maroc / Brésil") → affiché tel quel dans la parenthèse du bracket/liste KO.

Initialisation (one-shot depuis la console navigateur)

À lancer une seule fois après déploiement ou forceReset :

```
(async function() {
  var mapping = {};
  allMatches.filter(m => m.isFT && ESPN_ID_MAP[m.id]).forEach(function(m) {
    mapping[ESPN_ID_MAP[m.id]] = {
      espnId: ESPN_ID_MAP[m.id], matchKey: m.id,
      t1: m.t1, t2: m.t2, score: m.score, dayKey: m.dayKey
    };
  });
  var r = await fetch('https://wc2026.mguenoun.workers.dev/stats/init', {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ mapping, forceReset: true })
  });
  console.log('Init:', await r.json());
})();
```

```
// Pipeline complet (step2+step3 en boucle)
(async function runPipeline() {
  var base = 'https://wc2026.mguenoun.workers.dev';
  var h = { headers: { Origin: location.origin } };
  for (var i = 0; i < 200; i++) {
    var s = await fetch(base + '/stats/status', h).then(r => r.json());
    if (s.queue.pending.length === 0 && s.queue.processing.length === 0) {
      console.log('Termine', s.kv.matches, 'matches'); break;
    }
    if (s.queue.pending.length > 0) await fetch(base + '/stats/step2', h);
    await fetch(base + '/stats/step3', h);
    await new Promise(res => setTimeout(res, 500));
  }
})();
```

Reset KV ratings (si recalibration formule)

```
# Dans worker/ (Git Bash) – pipeline_queue via fichier (wrangler strip les guillemets en CLI)
echo '{"pending":[],"processing":[],"done":[]}' > /tmp/pq.json
npx wrangler kv key put --remote --binding=STATS_KV "pipeline_queue" --path /tmp/pq.json

# Supprimer les clés match:* une par une
npx wrangler kv key list --remote --binding=STATS_KV --prefix="match:"
npx wrangler kv key delete --remote --binding=STATS_KV "match:<id>"

# Relancer step1 puis pipeline
curl -H "Origin: http://localhost" https://wc2026.mguenoun.workers.dev/stats/step1
```

Monitoring

```
fetch('https://wc2026.mguenoun.workers.dev/stats/status')
  .then(r => r.json()).then(d => console.log(JSON.stringify(d, null, 2)));
```

Cloudflare Dashboard → worker `wc2026` → **Observability** → **Logs** : filtrer sur `scheduled` .

Structure du repo GitHub

```
mguenoun/wc2026 (repo)
├─ index.html          # Cloudflare Pages (main → prod, staging → recette)
├─ js/
│  ├─ config.js        v3 # USER_TZ, localTime(), liveMinute() avec utcDate
│  ├─ fallback.js      v3
│  ├─ api.js           v12 # fetchFairPlay(), utcDate dans processMatches(), yc/rc standings
│  ├─ predictions.js   v5 # Modèle Poisson v4 + prior FIFA
│  ├─ ratings.js       v4
│  ├─ modal.js         v7 # Lien YouTube par but
│  ├─ pitch.js         v4
│  ├─ map.js           v7 # Stade : openMap() uniquement (OSM + Google Maps)
│  ├─ render.js        v13 # KPI bar, renderFairPlay(), localTime() dans matchRow
│  ├─ rankings.js      v3 # Min cumulées + Moy. pondérée
│  └─ state.js         v8 # 8 vues (+ fairplay), KPI bar toggle, init fuseau horaire
├─ worker/
│  └─ worker.js        v7 # Deploye via wrangler deploy
├─ tests/
│  ├─ browser/
│  │  ├─ processMatches.test.js # normTeam, processMatches, computeStandings + pos
│  │  └─ render.test.js        # getAll3rd (ex aequo), resolveKOTeam, buildThirdAssign
│  └─ worker/
│     └─ calcRating.test.js
├─ docs/
│  ├─ diag1_architecture.{svg,png}
│  ├─ diag2_sequence.{svg,png}
│  ├─ diag3_interactions.{svg,png}
│  ├─ ARCHITECTURE.pdf
│  └─ gen_pdf.js          # node docs/gen_pdf.js → ARCHITECTURE.pdf
└─ ARCHITECTURE.md
```

Mise en place VSCode + Wrangler

wrangler.toml

```
name = "wc2026"
main = "worker/worker.js"
compatibility_date = "2026-06-11"

[[kv_namespaces]]
binding = "STATS_KV"
id = "749700d94d114156a758b14e2e9b6587"

[triggers]
crons = ["*/30 * * * *", "0 */2 * * *", "10 */2 * * *", "20 */2 * * *"]
```

Commandes utiles

```
wrangler deploy          # Deployer le worker
wrangler dev --local     # Dev local
wrangler tail            # Logs temps reel
wrangler secret put API_TOKEN_FD
```

URLs de production

Service	URL
Dashboard (prod)	https://wc2026.pages.dev
Dashboard (staging)	https://staging.wc2026.pages.dev
Worker	https://wc2026.mguenoun.workers.dev
Status pipeline	https://wc2026.mguenoun.workers.dev/stats/status
Classement joueurs	https://wc2026.mguenoun.workers.dev/stats/players
Classement gardiens	https://wc2026.mguenoun.workers.dev/stats/keepers

Diagrammes d'architecture

PNG dans `docs/` . Pour regenerer : `node docs/gen_diagrams.js`

Diagramme 1 — Composants et déploiement

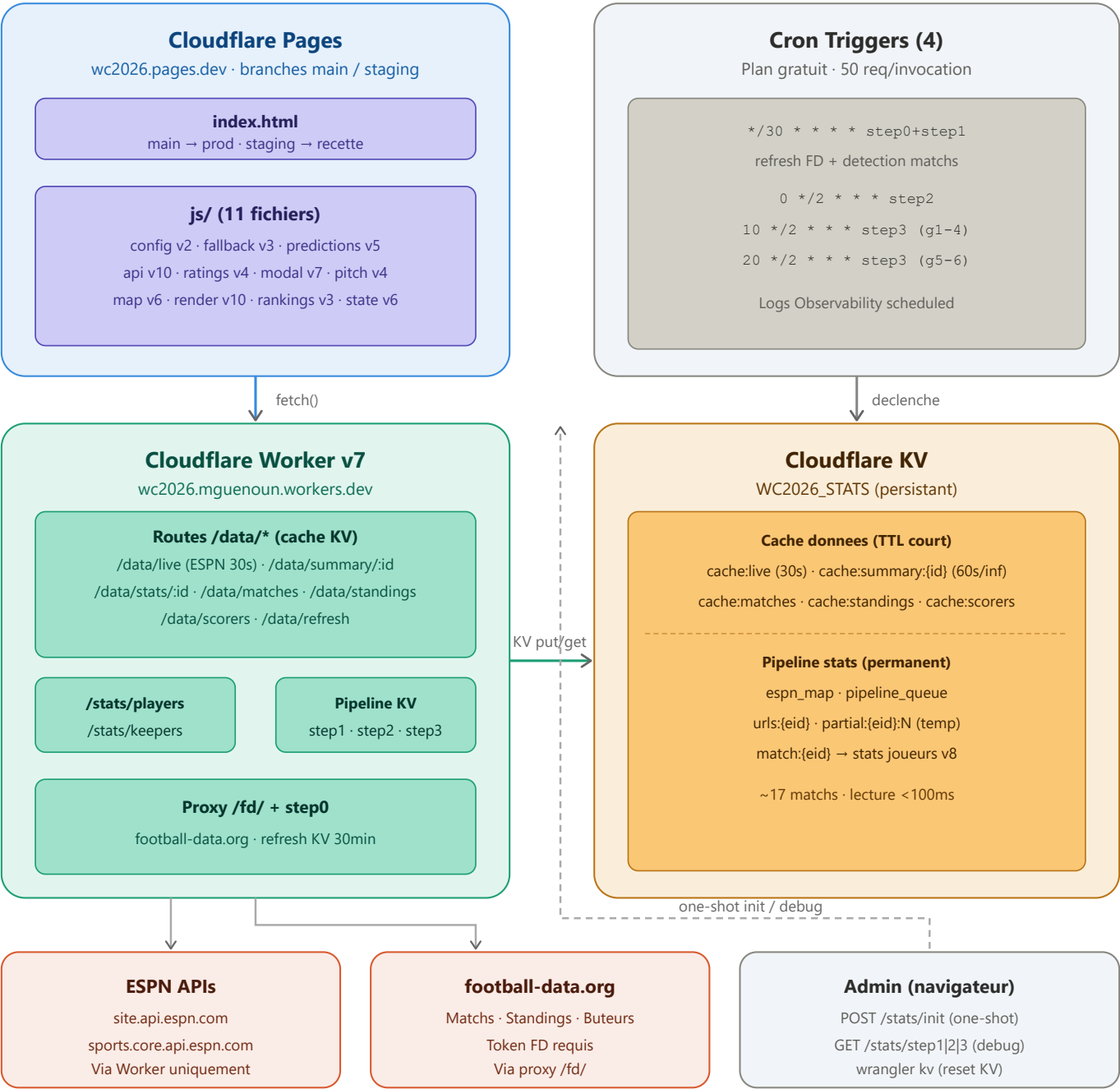
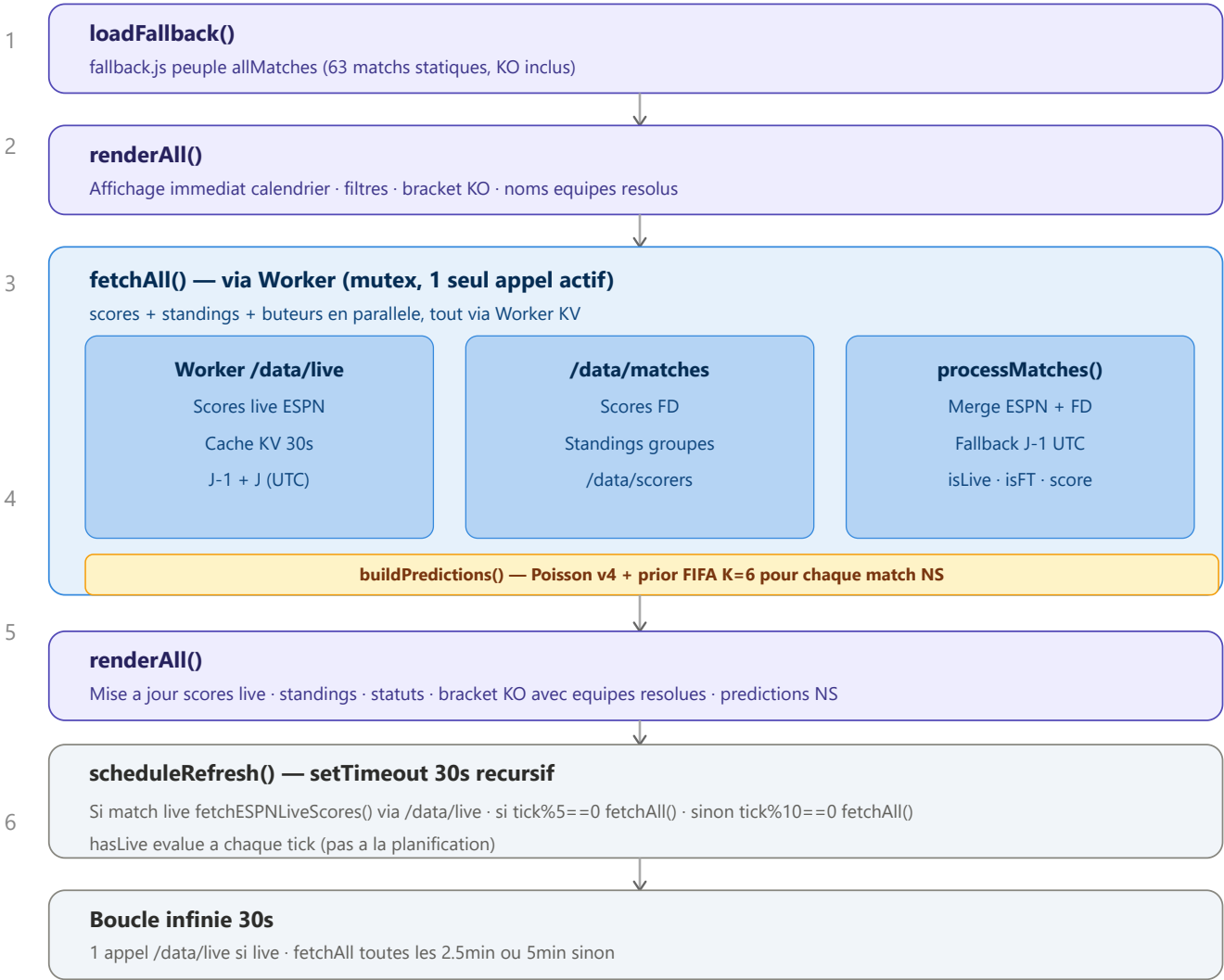


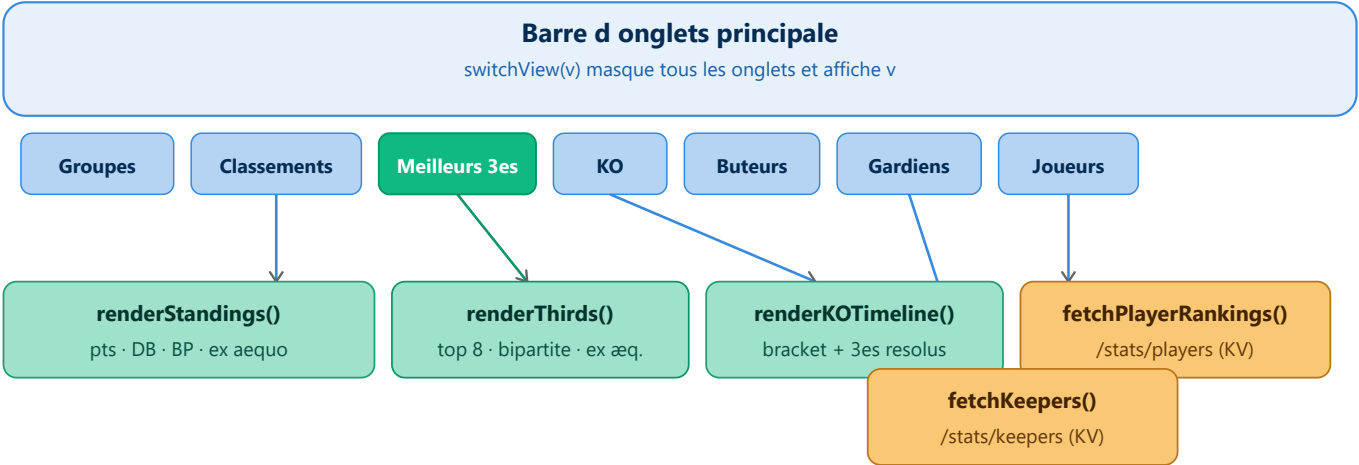
Diagramme 2 — Sequence au chargement de la page

Sequence au chargement de la page

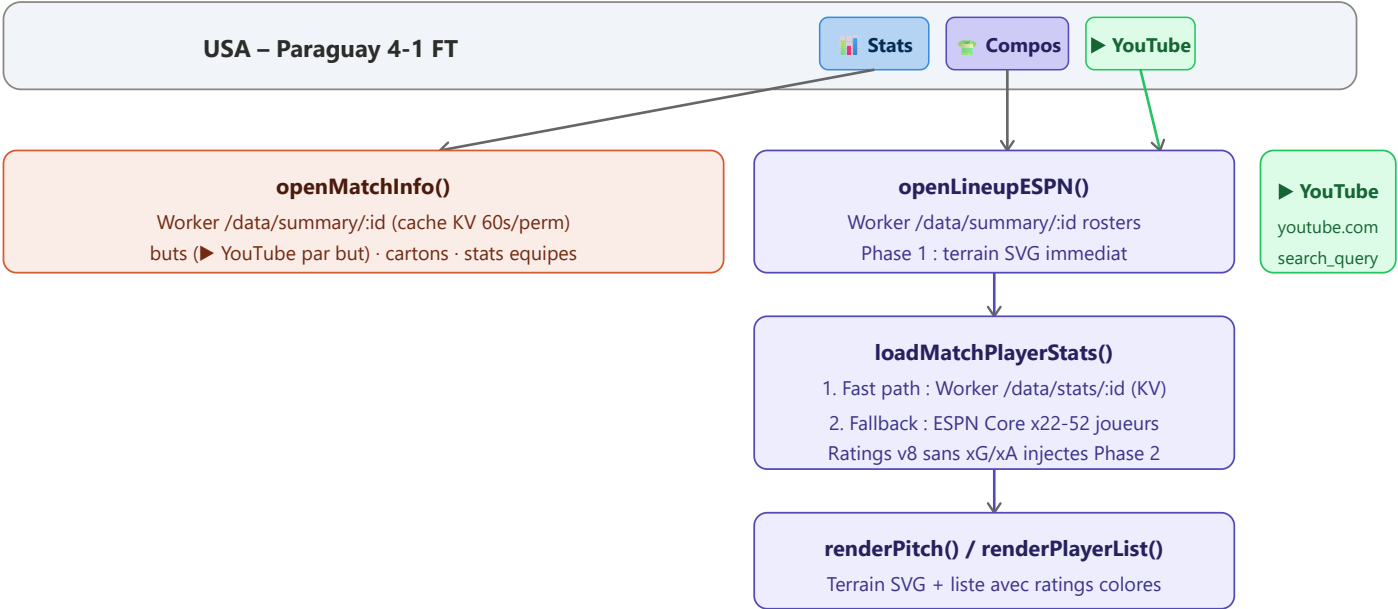
Executes dans state.js (charge en dernier)



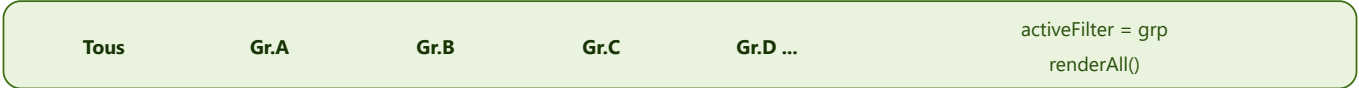
Interactions UI — actions declenchees par clic



Interactions sur un match (clic sur une ligne)



Filtres groupes



- Depuis memoire locale : allMatches / standings / scorers
- Lazy load unique Worker KV /stats/players ou /stats/keepers
- Worker /data/summary/:id cache KV (rapide des 2e appel)
- Lien YouTube direct (youtube.com/results?search_query=...)
- Ratings v8 : KV fast-path ou ESPN Core fallback (1-3s)